VLM Reliability Toolkit — Complete Code File Reference

VLM Reliability Toolkit — Complete Code File Reference

A detailed, file-by-file explanation of every source file in the project, organized by role in the pipeline.

Root-Level Entry Points & Utilities

`app.py` — Legacy Streamlit UI (396 lines)

A dark-themed Streamlit web interface that predates the Next.js dashboard. It provides:

- Image upload and clipboard paste
- VLM provider selection (sidebar dropdown)
- Candidate count slider (1–3)
- Reliability policy sliders (visual verification weight, agreement weight, accept/correct thresholds)
- “Run reliability analysis” button that calls **core.pipeline.run()**
- Tabbed results view:
- **Overview** — annotated image with bounding boxes plus the model response
- **Claim Inspector** — per-claim status, score, signals, and claim graph
- **Evidence** — bar chart of supported / contradicted / ambiguous claims
- **Candidates** — ranked comparison table
- **Correction** — original vs. corrected text side by side, JSON export
- Experiments page — SQLite history with JSON and CSV export

This is the original interface; **frontend_adapter.py** plus the Next.js frontend is the newer recommended one.

`vlm.py` — VLM Provider Adapters (167 lines)

The bridge to external image-language AI models. Defines:

- **VLMProvider** — abstract base class with a **generate(image_path, prompt)** method
- **OpenAICompatibleProvider** — handles OpenAI, Groq, and Llama (any OpenAI-compatible endpoint). Encodes the image as base64, sends it as an **image_url** message, returns **{response, model, provider, probability, probability_available, error}**. Note: probability is always <font

face='Courier'>**None** / **False** because these APIs do not expose native token logits.

- **GeminiProvider** — handles Google Gemini via the **google-genai** SDK, supporting both the older **interactions.create** and newer **models.generate_content** APIs.
- **providers()** / **provider_models()** — return the list of supported provider IDs and their default model IDs (safe for the UI; no API keys are leaked)
- **get_provider(name)** — resolves a configured provider by reading **.env** API keys. Returns **None** if credentials are missing (placeholder values like **your_api_key_here** are detected and treated as absent)
- **run_vlm_task()** / **generate_vlm_response()** — top-level functions that wrap provider selection with a safety prompt ("Answer concisely using only visible image content. Avoid unsupported details.")

Default models: OpenAI **gpt-4.1-mini**, Groq **qwen/qwen3.8-27b**, Gemini **gemini-3.8-flash**, Llama **meta-llama/llama-4-scout**

`vlm_engine.py` — VLM Execution Abstraction (34 lines)

Defines the **VLMEngine** protocol and **ProviderVLMEngine** implementation:

- **generate(image_path, query)** — calls the provider once
- **generate_candidates(image_path, query, n)** — calls the provider **n** times with slightly different prompts ("Candidate sample 1 of 3", etc.). Critically, this is explicitly *not* native n-best logits (which most APIs do not expose). Each result is tagged **"generation_strategy": "repeated_controlled_generation_not_native_nbest"** for audit honesty.
- **get_token_confidence(response)** — extracts **probability** if **probability_available** is true; otherwise returns **None**

`claims.py` — Legacy Claim Extractor (MVP, 54 lines)

A simple, backward-compatible claim extractor used by the older MVP path (**app.py**, **tests/test_core.py**). Contains:

- **COLORS** and **COMMON_OBJECTS** vocabularies (person, car, dog, surfboard, shirt, jeans, etc.)
- **ALIASES** (bike→bicycle, grey→gray, man/woman/child→person)

- **normalize_object(value)** — lowercases, strips, singularizes plurals, applies aliases
- **extract_claims(text)** — regex-based extraction of **color + object** pairs and bare object mentions. Returns a list of dicts: **[{"object": "car", "attribute": "color", "value": "red"}, ...]**

`correction.py` — Legacy Row-Based Correction (MVP, 21 lines)

A simpler text correction helper used by **tests/test_core.py** and **app.py**. Takes an original string plus a list of **{object, status, verified_value}** rows:

- **INCORRECT** + **verified_value** — regex-replaces the wrong color attached to that object (e.g., **"blue car"** → **"red car"**)
- **NOT VERIFIED** — removes the object phrase from the text
- Cleans up double spaces and trailing punctuation

`scoring.py` — MVP Scoring Policy (12 lines)

A simple threshold-based scorer for the older pipeline:

- **HIGH_THRESHOLD = 0.80**, **LOW_THRESHOLD = 0.40**
- **score_claims(rows)** — averages claim scores; returns **ACCEPT** (≥0.80), **CORRECT** (≤0.40), or **FLAG** (middle)

`utils.py` — Result Persistence Utility (18 lines)

save_result(payload, path) — appends a timestamped JSON record to **results/results.json**. Used by the legacy path; the newer pipeline uses SQLite via **database/db.py**.

`demo_paper_example.py` — Deterministic Paper Demo (25 lines)

Runs the paper's Table IV correction example without any API keys or model downloads:

- Hardcodes VLM text: **"A person in a blue shirt and red jeans stands on the beach holding a surfboard."**

- Manually assigns evidence: surfboard = **UNSUPPORTED** (no surfboard in image), shirt = **CONTRADICTED** (verified value: red), jeans = **CONTRADICTED** (verified value: blue)
- Runs the full reliability + correction pipeline and prints JSON output
- Expected corrected output: **"A person in a red shirt and blue jeans stands on the beach."**

`frontend_adapter.py` — Python HTTP API for Next.js Frontend (255 lines)

A standalone **http.server**-based API that the Next.js dashboard calls. Runs on port 8010 by default:

- **GET /health** — returns **{"status": "ok"}**
- **GET /api/config** — returns provider list, model IDs, default weights/thresholds from **config.yaml**
- **GET /api/history** — lists recent SQLite experiments
- **GET /api/history/<id>** — returns a specific experiment's full payload
- **POST /api/analyze** — accepts base64-encoded image + query + candidate count + weights + provider, runs **core.pipeline.run()**, saves to SQLite, returns analysis with claim graphs
- Handles chunked transfer encoding (Next.js proxy), CORS, image size limits (15 MB), and cleanup of temp files

---

`make_pdf.py` — PDF Generation Script

A ReportLab-based script that converts **VLM_Reliability_Toolkit_Explained.md** into a styled PDF.

Core Package

`core/schemas.py` — Typed Data Structures (63 lines)

Defines the core dataclasses that flow through the pipeline:

- **Claim** — **id**, **type** (object / attribute / relation / quantity), **subject**, **value**, **attribute**, **object**, **text**, **instance_id**. A stable ID is generated via SHA-1 hash of the claim key.
- **Evidence** — **claim_id**, **status** (SUPPORTED / CONTRADICTED / UNSUPPORTED / AMBIGUOUS), **score** (0–1), **source**, <font

face='Courier'>**confidence**, **boxes** (bounding boxes), **verified_value** (what the image actually shows), **explanation**, **evidence_quality**, **signals** (dict), **diagnosis** (e.g., **OBJECT_HALLUCINATION**, **ATTRIBUTE_BINDING_ERROR**), **binding_status**, **instance_ids**, **region_crop** (data URL for the cropped region)

- **Candidate** — **id**, **text** (the VLM's response), **model**, **claims**, **evidence**, **agreement**, **reliability**, **decision**, **corrected_text**, **diagnosis**, **probability**, **probability_available**, **claim_reliability** (per-claim scores), **generator_provider**, **verifier_provider**, **verifier_model**, **latency_ms**, **generation_strategy**, **stage_latency_ms**
- **Candidate.export()** — serializes everything to a dict for JSON storage and dashboard display

`core/pipeline.py` — Pipeline Orchestrator (51 lines)

The **run()** function that ties everything together. For each candidate:

1. Generates text from the VLM (or uses manual override)
2. Extracts claims via **claims.extractor.extract()**
3. Verifies claims via **verification.service.verify()**
4. Computes cross-candidate agreement via **reliability.engine.agreement()**
5. Scores reliability via **reliability.engine.score()**
6. Applies corrections via **correction.engine.correct()**
7. Sorts candidates by reliability (highest first)
8. Optionally upgrades the decision to **RERANK** if a lower-ranked candidate is substantially better (≥0.15 gap) and would be ACCEPTED

Returns **(candidates, error)**.

Claims Package

`claims/__init__.py` — Claim Vocabulary (16 lines)

Shared vocabulary used by both the legacy and new extractors:

- **COLORS** — 12 color names
- **COMMON_OBJECTS** — 28 object nouns (person, dog, car, bicycle, surfboard, shirt, jeans, hat, etc.)
- **ALIASES** — synonym normalization
- **normalize_object()** — singularization plus alias mapping
- **extract_claims()** — thin wrapper that converts **Claim** objects back to legacy dict format

`claims/extractor.py` — Conservative Claim Decomposition (41 lines)

The main claim extractor used by the production pipeline (**core/pipeline.py**). Converts free-text VLM output into **Claim** objects:

- **Attribute claims**: regex finds **color + object** pairs (e.g., “red car” → attribute claim: car is red)
- **Object claims**: finds any known object noun mentioned in the text
- **Quantity claims**: finds number words (“one”–“five”) plus object (e.g., “two cars” → quantity claim: 2 cars)
- **Relation claims**: finds spatial phrases (“next to”, “left of”, “right of”, “above”, “below”, “beside”, “near”) between two objects
- **Implied object claims**: if an attribute or quantity claim mentions an object that does not already have an object claim, one is auto-added (attribute existence implies object existence)
- Each claim gets a stable SHA-1-based ID (e.g., **C-a3f5b2c9d1**)

`claims/graph.py` — Claim Graph Structure (22 lines)

Converts a list of **Claim** objects into a graph (nodes plus edges) for dashboard visualization:

- **Nodes**: objects (e.g., “car”), attribute values (e.g., “red”), quantities, object instances
- **Edges**: labeled with the relationship (e.g., “color” from car → red, “next_to” from car → bicycle, “instance” from object class to specific instance)
- Returned as **{"nodes": [...], "edges": [...], "claims": [...]}** and consumed by the frontend to render a Graphviz chart

---

Verification Package

`verification/grounding.py` — Grounding DINO Adapter (78 lines)

Open-vocabulary object detection using Hugging Face **IDEA-Research/grounding-dino-tiny**:

- **GroundingResult** — dataclass with **object_name**, **boxes**, <font

face='Courier'>**scores**, **available**, **reason**, **backend**

- **Grounder** — protocol defining **detect(image_path, object_name)**
- **GroundingDINO** — lazy-loads the model on first call. Uses **AutoProcessor** + **AutoModelForZeroShotObjectDetection** from transformers. Post-processes detections with box and text thresholds. Returns boxes as **[[x1, y1, x2, y2], ...]** and confidence scores. If weights cannot be loaded (offline, missing dependency), returns **available=False** with an explicit reason — never fabricates detections.
- **local_files_only** config flag allows offline or reproducible runs after weights are cached

`verification/object_verifier.py` — Object Existence Verification (30 lines)

Implements the paper's equation **s_obj(o) = max_{b in B_o} d_b**:

- **ObjectVerifier.verify_object(object_name, image_path)** — calls Grounding DINO to detect the object
- If max detection score ≥ threshold → **SUPPORTED** with the max score
- If no detection meets threshold → **UNSUPPORTED** with diagnosis **UNSUPPORTED_CLAIM**
- If model unavailable → **AMBIGUOUS** with **evidence_quality=0.0**
- Returns an **Evidence** object with boxes, scores, and the grounding equation in **signals**

`verification/attribute_verifier.py` — Region-Level CLIP Attribute Probing (113 lines)

The key innovation: checks attributes **only on the cropped object region**, never the full image:

- **CLIPRegionProbe** — lazy-loads **openai/clip-vit-base-patch32**. **classify(crop, attribute_type, labels)** runs softmax over color prompts (e.g., "a red object", "a blue object") on the image crop. Only supports **color** attribute type; others return **AMBIGUOUS**.
- **crop_data_url(image_path, box)** — crops the image to a grounded box, resizes to ≤256 px, returns a base64 data URL for audit or display
- **AttributeVerifier.verify(image_path, boxes, box_scores, attribute_type, attribute_value, claim_id)**:

1. Crops each detected object box from the image
2. Runs CLIP on each crop to get color probabilities
3. Computes binding score: **s_attr(o,a) = max_b [d_b × P(a | crop_b)]** (detection confidence × crop color probability)
4. If best score ≥ threshold → **SUPPORTED**

5. If a different color scores ≥ threshold on the same crop → **CONTRADICTED** with **diagnosis=ATTRIBUTE_BINDING_ERROR** and **verified_value** = the actual color
6. Otherwise → **UNSUPPORTED**

- Never validates “blue car” just because blue appears elsewhere in the image

`verification/service.py` — Verification Service Orchestrator (91 lines)

The entry point **verify(image_path, claims)** called by the pipeline:

- Caches verifier instances (**@lru_cache**) — loads Grounding DINO plus CLIP once
- For each unique object subject across all claims, runs **ObjectVerifier.verify_object()** once (avoids redundant detections)
- Dispatches each claim type to the right verifier:
- **object** → object existence evidence
- **attribute** → **AttributeVerifier.verify()** on the grounded crop
- **quantity** → compares Grounding DINO box count to claimed count
- **relation** → checks bounding box geometry (left_of, right_of, above, below, next_to, near) between subject and object boxes
- Returns a list of **Evidence** objects, one per claim

`verification/fusion.py` — Evidence Fusion (38 lines)

fuse(claim_id, signals, explanation) — combines multiple named signals into a single evidence record:

- Each signal is either a raw score or **{"score": ..., "available": bool, "contributes_to_claim": bool}**
- Missing or unavailable signals are excluded from averaging (never treated as 0 or fabricated)
- Score = mean of available signals; evidence quality = fraction of available signals
- Status determined by signal distribution: positive (≥0.75) → SUPPORTED; negative (≤0.25) → CONTRADICTED; both → AMBIGUOUS; neither → UNSUPPORTED

Reliability Package

`reliability/engine.py` — Scoring + Decision Logic (69 lines)

The core scoring and decision engine:

- **agreement(candidates)** — measures cross-candidate claim agreement. Normalizes each claim to a key **(type, subject, attribute, value, object,**

instance_id), counts how many candidates share each claim, averages the fraction. High agreement means the answer is not a fluke.

- **score(candidate, weights, high, low)** — computes the 0–1 reliability score:
- Per-claim reliability: AMBIGUOUS or missing → 0.5; SUPPORTED / CONTRADICTED / UNSUPPORTED → evidence score
- Visual verification score = weighted mean of non-ambiguous evidence scores × evidence quality
- Token probability: if unavailable, its configured weight is redistributed to visual verification (no fake numbers)
- Final: **visual_score = (verification×wv + probability×wp + agreement×wa) / (wv+wp+wa)**, clamped to [0,1]
- **Decision logic** (conservative priority order):

1. **CORRECT** — any high-quality (evidence_quality ≥ 0.8) CONTRADICTED evidence with a clear diagnosis, OR high-quality UNSUPPORTED object claim with low score
2. **ABSTAIN** — any AMBIGUOUS or UNSUPPORTED claim, OR low reliability with no evidence
3. **ACCEPT** — reliability ≥ high threshold (0.80) AND nothing contradicted
4. Otherwise → ABSTAIN

Correction Package

`correction/__init__.py` — Legacy Row-Based Correction Helper (15 lines)

Same as root-level **correction.py** — a backward-compatible function for the MVP tests and legacy UI.

`correction/engine.py` — Surgical Text Correction Engine (32 lines)

Applies evidence-backed corrections to a candidate's text using regex:

- **OBJECT_HALLUCINATION** (CONTRADICTED) — removes the object phrase (e.g., **"holding a surfboard"** → deleted)
- **ATTRIBUTE_BINDING_ERROR** (CONTRADICTED + verified_value) — swaps the wrong color for the verified one (e.g., **"blue car"** → **"red car"**)
- **QUANTITY_ERROR** (CONTRADICTED + verified_value) — fixes the count (e.g., **"three dogs"** → **"two dogs"**)
- **RELATION_ERROR** — logged but not auto-corrected (too risky to rewrite spatial relationships)
- **AMBIGUOUS / UNSUPPORTED** — logged as diagnosis but **never** corrected (conservative: absence of evidence is not evidence of error)
- If no correction applies, **corrected_text** equals the original and diagnosis is **["NO_STRONG_ERROR"]**

- Cleans up double spaces and trailing punctuation

Evaluation Package

`evaluation/metrics.py` — Binary Metrics + Calibration (52 lines)

Computes metrics from **labelled records only** (no synthetic scores implied):

- _**binary_metrics(labels, predictions)** — accuracy, precision, recall, F1
- **classification_metrics(records)** — requires **expected_reliable** (bool) and **reliability** (0–1) fields:
- Brier score, sample count
- Expected Calibration Error (ECE) via 10-bin reliability diagram — requires ≥30 samples, otherwise returns **calibration_available=False**
- Error-type detection stats (if **error_type** / **predicted_error_type** fields present)
- Abstention rate (fraction of ABSTAIN decisions)
- Correction success rate and false correction rate (if **corrected_correct** field present)

`evaluation/evaluator.py` — Benchmark Adapters (71 lines)

Dataset-agnostic harnesses that consume prepared JSONL files (no auto-downloading):

- **pope(records)** — POPE yes/no object existence by split (random / popular / adversarial): accuracy, precision, recall, F1, yes-ratio
- **chair(records)** — CHAIR from **ground_truth_objects** / **mentioned_objects** sets: CHAIR-S (sentence-level hallucination), CHAIR-I (instance-level), object recall
- **winoground(records)** — Winoground: text score, image score, group score from **text_correct** / **image_correct** fields
- **sugarcrepe(records)** — SugarCrepe: replace / swap / add accuracy from **category** / **correct** fields
- **run_all(dataset_dir)** — loads all four **.jsonl** files from a directory and runs each adapter

`evaluation/run_benchmarks.py` — CLI Entry Point (9 lines)

python -m evaluation.run_benchmarks --data data/benchmarks --out results/report.json — runs all benchmarks and writes a JSON report.

`evaluation/reports.py` — Report Persistence (12 lines)

write_report(result, path, configuration, seed) — saves a JSON report with timestamp, seed, configuration, environment info (Python version, platform), and results. Never fabricates unrun metrics.

`evaluation/ablations.py` — Ablation Configurations (8 lines)

Named experiment configurations for comparing system variants:

- **base_vlm** — no verification, no attribute binding, 1 candidate, no reliability scoring
- **full_toolkit** — verification plus attribute binding plus 3 candidates plus reliability scoring
- **no_attribute_binding** — verification only, no region-level CLIP
- **no_reliability_scoring** — verification plus attribute binding, but no reliability score
- **n_equals_1** — full toolkit but only 1 candidate (no agreement signal)

Database Package

`database/db.py` — SQLite Experiment Storage (11 lines)

- **DB** — **results/experiments.db**
- **init()** — creates the **experiments** table (id, created_at, query, payload JSON)
- **save(query, payload)** — inserts an experiment record, returns the auto-increment ID
- **recent(limit)** — returns the most recent experiments (id, created_at, query, payload)

Used by both **frontend_adapter.py** and **app.py** for audit history and the Experiments dashboard page.

Frontend

`frontend/` — Next.js Dashboard

A Next.js web application (TypeScript, Tailwind) at **frontend/**:

- **frontend/package.json** — dependencies (Next.js, React, etc.)
- **frontend/tsconfig.json** — TypeScript configuration
- **frontend/pnpm-lock.yaml** — lockfile
- Communicates with **frontend_adapter.py** on port 8010 via REST API

- Provides the modern dashboard interface described in **app.py** but with a React frontend

---

Tests

`tests/test_core.py` — Legacy MVP Unit Tests (29 lines)

Tests for the older **claims.py**, **correction.py**, **scoring.py** modules:

- Color plus object extraction from text
- Threshold decisions (ACCEPT / CORRECT / FLAG)
- Removing unverified objects from text
- Replacing wrong colors with verified values

`tests/test_toolkit.py` — Toolkit Unit Tests (14 lines)

Tests for the new pipeline modules:

- Quantity and relation claim extraction
- Claim-level agreement (identical claims → agreement 1.0) and reliability scoring

`tests/test_research_reliability.py` — Research Reliability Unit Tests (69 lines)

Comprehensive tests using fake verifiers (no model downloads needed):

- Object score = max grounding confidence (**s_obj = max d_b**)
- Missing grounding → UNSUPPORTED, score 0
- Attribute binding uses object crop plus equation **s_attr = max d_b × P(a|crop_b)**; crop data URL is a base64 JPEG
- Unavailable visual backend → AMBIGUOUS, evidence_quality 0
- Binding correction: CONTRADICTED attribute → CORRECT decision, verified color in output
- Unavailable token probability → weight redistributed to verification
- Calibration refuses tiny samples (<30)

Configuration

`config/config.yaml` — Configuration File (24 lines)

All tunable parameters:

`yaml

vlm:

provider: gemini # openai | groq | gemini | llama

model: gemini-2.5-flash

candidate_count: 3

reliability:

lambda_verification: 0.6 # visual evidence weight

lambda_probability: 0.2 # token confidence weight (redistributed if unavailable)

lambda_agreement: 0.2 # cross-candidate agreement weight

high_threshold: 0.70 # accept threshold

low_threshold: 0.30 # correct / abstain threshold

verification:

method: grounding_dino_plus_region_probe

grounding_model: IDEA-Research/grounding-dino-tiny

grounding_threshold: 0.30

attribute_model: openai/clip-vit-base-patch32

attribute_threshold: 0.50

local_files_only: false

generation:

num_candidates: 3

temperature: 0.7

decision:

tau_lo: 0.30

tau_hi: 0.70

`

Pipeline Summary (How Files Connect)

`

User uploads image + query

■

▼

frontend_adapter.py (HTTP API) ■■■■■■■■■■■■■■■■■

■ ■

▼ ■

core.pipeline.run() ■

■ ■

■■■ vlm_engine.py ■■■■ vlm.py ■■■■ External VLM API (candidate generation)

■

■■■ claims.extractor.extract() ■■■■■■■■■■ Claim objects (atomic, typed)

■

■■■ verification.service.verify() ■■■■■■■■■ verification.grounding.GroundingDINO

■ ■ (open-vocab object boxes)

■ ■■■■ verification.object_verifier.ObjectVerifier

■ ■ (s_obj = max confidence → SUPPORTED/UNSUPPORTED/AMBIGUOUS)

■ ■■■■ verification.attribute_verifier.CLIPRegionProbe

■ (s_attr = d_b × P(a|crop_b) on object crop)

■

■■■ reliability.engine.agreement() ■■■■■■ Cross-candidate claim agreement

■■■ reliability.engine.score() ■■■■■■■■■ 0–1 reliability score + 4-way decision

■■■ correction.engine.correct() ■■■■■■■■ Surgical text fixes (color swap, object removal)

■

▼

database.db.save() ■■■■ SQLite audit record (image hash, candidates, evidence, decisions)

■

▼

Next.js dashboard / Streamlit UI ■■■■ Human-readable results with bounding boxes

`

Each file has a single, well-defined responsibility, and the pipeline is designed so that no component fabricates evidence — if a visual model is unavailable, the system reports **AMBIGUOUS** rather than guessing.